Ember Attractor: Black Hole

Niagara at Scale | One-page proposal | UE 5.7 | Individual project | Custom variation of Track B

Direction and scope

A one-room particle game built from the two gravity effects in our *Gravity* project: an explosion of hot embers and a black hole that swallows them. The player steers the black hole; the embers are the resource. Everything is rebuilt independently here. No Gravity assets or code are imported, and no existing integration is claimed. This replaces the Week 1 "guide embers into a recovery zone" idea, because the black hole gives a tighter two-way loop. The Week 1 particle systems, controls and capture tools carry over unchanged.

Mechanic

An explosion scatters hot embers and debris across a chamber. The mouse moves a black hole: anything inside its event horizon is swallowed for free. Holding the mouse button pulls matter in from a wider radius and spins it into an accretion swirl, but this drains a limited **energy** tank that only recharges while the player lets go. Embers **cool** over time and lose value. The goal is to reach critical mass before the timer ends, which means trading energy now against matter that is going cold. Swallowed mass grows the horizon and strengthens the pull, so the black hole visibly feeds on the simulation.

Two ends of the cost curve

Simple and many: 100,000+ GPU sprite embers with three rules: spring pull to the hole, vortex swirl and drag. **Complex and few:** 500–5,000 CPU mesh debris with the same forces plus scene collision, age-driven heat, and a per-particle state machine: Drifting → Captured (latched with hysteresis, glows cyan, orbits without costing energy) → Swallowed. Both systems feed the same gameplay readback. Runtime count control: `ember.GPUCount`, `ember.CPUCount` and preset keys.

Gameplay data path

Gameplay → Niagara: user parameters for hole position, pull strength, pull radius, vortex, horizon radius and cooling time. **Niagara → gameplay:** an *Export Particle Data* interface writes a particle only in the frame it crosses the horizon, sending its age. A *Kill Particles* module consumes the same flag, so each particle is reported exactly once (asynchronous readback on the GPU, synchronous on the CPU). C++ turns age into heat and heat into mass, which feeds back into the horizon and the pull. That makes the coupling two-way and makes the outcome depend on particle state.

Budget and measurement

Budget stated before building: 1080p at 60 fps (16.6 ms per frame), with combined Niagara game- and render-thread cost at or below 4 ms; GPU time is recorded separately. Sweep each system across at least five counts until the budget fails (GPU 100k→8M, CPU 500→20k), with `stat Niagara`, `stat GPU`, the Niagara Debugger, engine CSVs and Unreal Insights. The optimization will be measured before and after on identical workloads: fixed vs dynamic bounds, and a CPU-to-GPU port of the debris.

References and milestones

Primary reference: Noita (played): loose, simulated material moved by forces rather than by a controlled character, and a few cells of the right material deciding the outcome. **Secondary: PixelJunk Shooter** (watched, about an hour of the first two episodes): water, lava and gas interactions as the puzzle, state changes read through colour, steam and sound, and the ship's heat gauge as the central resource. Full observations: Documentation/ReferenceNotes.md.

Week 1: both systems in a test map, runtime count control, first capture (done). **Week 2:** black-hole round with energy and cooling, verified two-way readback, full scaling sweep. **Week 3:** measured optimization, two-page report, 60–90 second video, tagged release.

Sources: Noita official site · PixelJunk Shooter Ultimate developer article